Guide : Création d'un Dashboard Interactif avec D3.js

Par Robert DIASSÉ

Ce guide constitue une perspective avancée sur les cours précédents en visualisation de données avec D3.js. Nous allons explorer la création d'un **dashboard interactif** combinant plusieurs graphiques pour offrir une vue synthétique et interactive de données complexes.

---

1. **Objectifs**

- Créer un dashboard regroupant plusieurs graphiques (barres, lignes, camemberts, etc.).
- Ajouter des interactions utilisateur (filtrage, survol, sélection, zoom).
- Organiser les composants pour une interface claire et intuitive.
- Intégrer des transitions fluides et des animations entre différents états des graphiques.
- Ajouter des légendes interactives pour améliorer la lisibilité.

---

2. **Préparation du projet**

Outils et bibliothèques

1. **D3.js** (pour les graphiques interactifs)
2. **HTML/CSS** (pour le design)
3. **JavaScript** (pour l'interactivité)
4. **TopJSON ou GeoJSON** (si vous incluez des cartes géographiques)

Structure du projet

Organisez votre projet comme suit :

```
project-directory/
|-- index.html
|-- styles.css
|-- scripts/
|   |-- dashboard.js
|   |-- utils.js
|-- data/
    |-- data.json
    |-- map.geojson
```

Ajoutez un dossier `assets/` si vous utilisez des images ou des icônes.

---

3. **Construction du Dashboard**

3.1. Structure HTML de base

Créez un fichier `index.html` pour structurer votre dashboard.

```html
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Dashboard Interactif</title>
    <link rel="stylesheet" href="styles.css">
    <script src="https://d3js.org/d3.v7.min.js"></script>
    <script defer src="scripts/dashboard.js"></script>
</head>
<body>
    <header>
        <h1>Dashboard Interactif</h1>
    </header>

    <div id="dashboard">
        <div id="filters">
            <!-- Filtres interactifs -->
            <label for="category">Choisir une catégorie :</label>
            <select id="category">
                <option value="all">Toutes</option>
            </select>
        </div>

        <div id="charts">
            <div id="chart1" class="chart"></div>
            <div id="chart2" class="chart"></div>
            <div id="chart3" class="chart"></div>
        </div>
    </div>

    <footer>
        <p>Créé avec D3.js</p>
    </footer>
</body>
</html>
```

3.2. Fichier CSS : `styles.css`

Ajoutez un design minimaliste et adaptatif.

```style
body {
    font-family: Arial, sans-serif;
    margin: 0;
    padding: 0;
    background-color: #f4f4f9;
    color: #333;
}

header {
    background-color: #4CAF50;
    color: white;
    padding: 1rem;
    text-align: center;
}

#dashboard {
    display: grid;
    grid-template-columns: 1fr 3fr;
    gap: 1rem;
    margin: 1rem;
}

#filters {
    background: white;
    padding: 1rem;
    border-radius: 5px;
    box-shadow: 0 2px 4px rgba(0, 0, 0, 0.1);
}

#charts {
    display: grid;
    grid-template-columns: repeat(auto-fit, minmax(300px, 1fr));
    gap: 1rem;
}

.chart {
    background: white;
    padding: 1rem;
    border-radius: 5px;
    box-shadow: 0 2px 4px rgba(0, 0, 0, 0.1);
}

footer {
    text-align: center;
    padding: 1rem;
    background-color: #4CAF50;
    color: white;
    position: fixed;
    bottom: 0;
    width: 100%;
}
```

3.3. Script JavaScript : `dashboard.js`

1. **Chargement des données**

```
const dataPath = "data/data.json";

async function loadData() {
    const data = await d3.json(dataPath);
    return data;
}
```

2. **Initialisation du Dashboard**

```
loadData().then(data => {
    // Initialiser les filtres
    initFilters(data);

    // Créer les graphiques
    createBarChart(data, "#chart1");
    createLineChart(data, "#chart2");
    createPieChart(data, "#chart3");
});
```

3. **Filtres dynamiques**

```
function initFilters(data) {
    const categories = Array.from(new Set(data.map(d => d.category)));
    const select = d3.select("#category");

    categories.forEach(cat => {
        select.append("option")
              .attr("value", cat)
              .text(cat);
    });

    select.on("change", () => {
        const selectedCategory = select.property("value");
        updateCharts(data, selectedCategory);
    });
}
```

4. **Graphiques (exemple)**

a. Diagramme à barres

```
function createBarChart(data, selector) {
    const svg = d3.select(selector)
                  .append("svg")
                  .attr("width", 400)
                  .attr("height", 300);

    const xScale = d3.scaleBand()
                     .domain(data.map(d => d.name))
                     .range([0, 400])
                     .padding(0.1);

    const yScale = d3.scaleLinear()
                     .domain([0, d3.max(data, d => d.value)])
                     .range([300, 0]);

    svg.selectAll("rect")
       .data(data)
       .enter()
       .append("rect")
       .attr("x", d => xScale(d.name))
       .attr("y", d => yScale(d.value))
       .attr("width", xScale.bandwidth())
       .attr("height", d => 300 - yScale(d.value))
       .attr("fill", "#4CAF50");

    // Axes
    svg.append("g")
       .attr("transform", "translate(0, 300)")
       .call(d3.axisBottom(xScale));

    svg.append("g")
       .call(d3.axisLeft(yScale));
}
```

---

b. Diagramme à lignes

```
function createLineChart(data, selector) {
    const svgWidth = 400;
    const svgHeight = 300;
    const margin = { top: 20, right: 20, bottom: 30, left: 50 };
    const width = svgWidth - margin.left - margin.right;
    const height = svgHeight - margin.top - margin.bottom;

    const svg = d3.select(selector)
                  .append("svg")
                  .attr("width", svgWidth)
                  .attr("height", svgHeight)
                  .append("g")
                  .attr("transform", `translate(${margin.left}, ${margin.top})`);
```

```
    const xScale = d3.scaleTime()
                    .domain(d3.extent(data, d => new Date(d.date)))
                    .range([0, width]);

    const yScale = d3.scaleLinear()
                    .domain([0, d3.max(data, d => d.value)])
                    .range([height, 0]);

    const line = d3.line()
                  .x(d => xScale(new Date(d.date)))
                  .y(d => yScale(d.value));

    svg.append("path")
       .datum(data)
       .attr("fill", "none")
       .attr("stroke", "#4CAF50")
       .attr("stroke-width", 2)
       .attr("d", line);

    // Axes
    svg.append("g")
       .attr("transform", `translate(0, ${height})`)
       .call(d3.axisBottom(xScale).ticks(5));

    svg.append("g")
       .call(d3.axisLeft(yScale));
}
```

---

c. Diagramme circulaire (camembert)

```
function createPieChart(data, selector) {
    const width = 300;
    const height = 300;
    const radius = Math.min(width, height) / 2;

    const svg = d3.select(selector)
                  .append("svg")
                  .attr("width", width)
                  .attr("height", height)
                  .append("g")
                  .attr("transform", `translate(${width / 2}, ${height / 2})`);

    const color = d3.scaleOrdinal(d3.schemeCategory10);

    const pie = d3.pie().value(d => d.value);
    const arc = d3.arc()
                  .innerRadius(0)
                  .outerRadius(radius);
```

```
    svg.selectAll("path")
       .data(pie(data))
       .enter()
       .append("path")
       .attr("d", arc)
       .attr("fill", d => color(d.data.name));
}
```

---

5. Mise à jour dynamique des graphiques

```
function updateCharts(data, category) {
    const filteredData = category === "all" ? data : data.filter(d => d.category
=== category);

    // Mettre à jour le diagramme à barres
    d3.select("#chart1 svg").remove();
    createBarChart(filteredData, "#chart1");

    // Mettre à jour le diagramme à lignes
    d3.select("#chart2 svg").remove();
    createLineChart(filteredData, "#chart2");

    // Mettre à jour le diagramme circulaire
    d3.select("#chart3 svg").remove();
    createPieChart(filteredData, "#chart3");
}
```

---

6. Données d'exemple

Ajoutez un fichier `data/data.json` contenant des données simulées :

```
[
    { "name": "Produit A", "value": 30, "category": "Catégorie 1", "date": "2025-
01-01" },
    { "name": "Produit B", "value": 50, "category": "Catégorie 2", "date": "2025-
01-02" },
    { "name": "Produit C", "value": 20, "category": "Catégorie 1", "date": "2025-
01-03" },
    { "name": "Produit D", "value": 40, "category": "Catégorie 2", "date": "2025-
01-04" }
]
```

---

7. Conclusion

Avec ce guide, vous aurais appris à créer un **dashboard interactif complet** avec D3.js. Ce projet peut servir de base pour des visualisations avancées et personnalisées. Les éléments clés à retenir sont :

1. La modularité du code pour faciliter les ajouts futurs.
2. L’importance des interactions utilisateur pour améliorer l’expérience.
3. L’utilisation de données réelles pour refléter des situations pratiques.

Explorez d'autres types de graphiques et essayez d'ajouter des cartes géographiques pour aller plus loin dans vos dashboards !